Credit Card Default Prediction:
A Cost-Sensitive Machine Learning Approach

Student Name	Ngoc Vu Nguyen
Student ID	24577999
Subject	Advanced Analytics Algorithm Assignment (A2)
Institution	University of Technology Sydney (UTS)
Semester	Autumn 2026
Option	Option 2 : Build a Machine Learning System to Address a Practical Challenge

Notebook Implementation Link on Google Colab (publicly accessible):

https://colab.research.google.com/drive/1tgfe6RKsGVu3mSeMvxmUpIHnuH4nAwlt?usp=sharing

---

This journal documents the design, implementation, and evaluation of a cost-sensitive LightGBM classifier for bank credit card default prediction, in which the training objective itself is modified to reflect empirically derived misclassification costs. Drawing on Federal Reserve (2024) and ECB Working Paper 2920 data, a 7.56× FN-vs-FP cost asymmetry is quantified from observable market statistics. A custom gradient-level objective function embeds this asymmetry directly into LightGBM's boosting loop, yielding lower expected financial cost than post-hoc threshold calibration alone.

1. Link to Implementation

The complete implementation is hosted on Google Colab and is publicly accessible via the link provided on the cover page. The notebook is self-contained and includes all of the following:

- Environment setup and library installation (pandas, numpy, scikit-learn, lightgbm, imbalanced-learn,matplotlib)
- Data loading pipeline, the dataset is loaded from Google Drive or a direct download link
- All preprocessing, feature engineering, and SMOTE oversampling steps
- Model training, hyperparameter tuning via GridSearchCV, and evaluation
- Cost-based threshold optimisation with visualisations

Note: The offline version of the notebook has been tested to match the online archived version on Google Colab. The notebook must be run in sequence from top to bottom; runtime is approximately 8–12 minutes on a standard Colab CPU instance due to GridSearchCV hyperparameter search across 7 models

2. Project Report

2.1 Problem Definition

2.1.1 Motivation and Background

Credit card default is a critical risk management problem for retail banks. When a bank issues credit, it bears two types of financial risk: the cost of credit loss when it approves a customer who subsequently defaults (false negative), and the opportunity cost of revenue foregone when it declines a customer who would have repaid (false positive). These costs are highly asymmetric, and this asymmetry drives the central design challenge of this project.

Traditional credit scoring approaches such as logistic regression scorecards or rules-based cutoff systems which are insufficient for this problem for three reasons: (1) they assume linear separability in a problem characterised by complex, non-linear feature interactions; (2) they are not designed to handle severe class imbalance (8.2% default rate in this dataset); and (3) they optimise for accuracy rather than business cost, failing to account for the asymmetric cost structure.

2.1.2 Formal Task Specification

Following the formal I/O specification framework, the task is defined at two phases:

Phase	Component	Specification	Format
Training	Input X	Feature matrix of customer demographics and financial history after preprocessing	$x \in R^{16}$ per record
Training	Output	Learned model parameters θ defining the conditional probability function P(default=1 | x; θ)	LightGBM ensemble of T=300 trees
Deployment	Input x	Single customer feature vector (same 16 features as training, after preprocessing)	$x \in R^{16}$
Deployment	Output	Binary decision and calibrated probability: approve (0) or decline (1), plus P(default=1 | x)	$\hat{y} \in \{0,1\}, \hat{p} \in [0,1]$

Table 1: Formal I/O specification for training and deployment phases.

2.1.3 Input Feature Description

Feature	Type	Description	Role
default_in_last_6months	Binary	Prior default flag within 6-month window	Strongest predictor (r=0.773)
prev_defaults	Integer	Count of historical defaults	2nd strongest (r=0.770)
credit_score	Continuous	Credit bureau score	Neg. correlation (r=-0.554)
net_yearly_income	Continuous	Annual net income (USD)	Financial capacity proxy
age (days_birth)	Continuous	Customer age in days (negated)	Demographic

days_employed	Continuous	Employment duration in days (cleaned)	Stability proxy
debt_to_income_ratio*	Engineered	credit_amount / net_yearly_income	Leverage indicator
children_family_ratio*	Engineered	total_children / total_family_members	Dependency burden
income_per_family_member*	Engineered	net_yearly_income / total_family_members	Adjusted income
gender, income_type, education_type, family_status, occupation_type, housing_type	Categorical (encoded)	Demographic categories	Behavioural context

*Table 2: Input features after preprocessing and engineering. * = engineered feature.*

2.1.4 Research Question

This project addresses the following research question:

> *Can a boosting classifier whose training loss directly encodes the empirical FN/FP cost asymmetry (custom cost-sensitive objective) achieve lower expected financial cost than a standard log-loss-trained classifier with post-hoc cost-based threshold calibration?*

2.1.5 Empirical Cost Grounding

The asymmetric cost structure is derived from observable financial data, not assumed. Two independent sources are used:

Cost Type	Value	Derivation	Source
False Negative C_{FN} Approve a defaulter	**$1,922.58**	Federal Reserve net return on assets: 3.87% of median credit limit $49,680	Federal Reserve Statistical Release G.19 (2024)
False Positive C_{FP} Decline a good customer	**$253.74**	ECB LGD opportunity cost rate: 0.59% of median credit limit $43,000	ECB Working Paper Series No. 2920 (2024)
Cost Ratio C_{FN} / C_{FP}	**7.56 : 1**	A missed defaulter costs 7.56x more than a wrongly declined good customer	Derived

Table 3: Empirically derived asymmetric misclassification costs.

2.2 Machine Learning Approach

2.2.1 System Overview — Pipeline

Stage	Operation	Justification
1. Data Ingestion	Load CSV (37,430 records × 19 features)	Real-world, non-trivial dataset; no toy benchmark
2. Cleaning	Median imputation (numerical); mode imputation (categorical); remove invalid employment record	Median robust to right-skewed income distributions; removes data-entry error (365,241 days employed)
3. Feature Engineering	Create debt_to_income_ratio, children_family_ratio, income_per_family_member	Domain-motivated leverage and dependency features
4. Feature Selection	Drop credit_limit, total_family_members, owns_house (RF importance + Pearson)	Reduces multicollinearity; retains engineered versions
5. Encoding	Label-encode all 8 categorical features	Required for tree-based models; preserves cardinality
6. Train/Test Split	Stratified 80/20 split	Preserves 8.2% minority class in both sets
7. Scaling	StandardScaler on numerical features	Required for SMOTE Euclidean distance; enables fair SVM/KNN comparison
8. SMOTE	Oversample minority class (training set only): 2,605 → 27,339	Applied post-split to prevent data leakage; k-NN interpolation in feature space
9. Model Training	GridSearchCV (5-fold, ROC-AUC scoring) across 7 models	Comprehensive algorithm comparison from linear to ensemble
10. Threshold Tuning	Cost-based threshold sweep on LightGBM predictions ($t \in [0.01, 0.99]$)	Aligns decision boundary with asymmetric business cost objective
11. Algorithmic Customisation	Cost-weigted gradient on tree-building process	Model prioritises splits that reduce error on defaulters

Table 4: Full data processing and modelling pipeline.

2.2.2 Why LightGBM — Theoretical Justification

LightGBM is a gradient boosting decision tree (GBDT) framework that extends standard GBDT with two key algorithmic innovations: Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB).

Gradient Boosting Foundation. The ensemble is constructed iteratively: $F_T(x) = F_0(x) + \eta \sum_{t=1}^{T} h_t(x)$ where each weak learner h is a decision tree fitted to the negative gradient of the loss with respect to the t current model output. For example, the residuals of the previous prediction. For binary classification, the loss is binary cross-entropy:

```
L(y, p) = -[y · log(p) + (1-y) · log(1-p)]
```

GOSS (Gradient-based One-Side Sampling). Standard GBDT samples all instances equally in each boosting round. GOSS retains all instances with large gradients (large training error) and randomly samples a fraction of small-gradient instances. This is particularly beneficial for imbalanced data: minority class instances (defaulters) naturally have larger gradients after early boosting rounds, so GOSS preferentially attends to them without explicit resampling; thus, complementing SMOTE's synthetic oversampling.

Leaf-wise Growth. Unlike level-wise (depth-first) trees in XGBoost, LightGBM grows trees leaf-wise, always expanding the leaf with the highest potential loss reduction. This produces more asymmetric, deeper trees that capture the complex non-linear interactions expected in credit behaviour data.

Property	Standard GBDT	LightGBM (GOSS +EFB)	Relevance to Task
Tree growth	Level-wise	Leaf-wise	Captures non-linear credit interactions
Sampling	All instances	High-gradient + random sample	Preferential minority class attention
Feature handling	All features	Exclusive Feature Bundling	Efficient with correlated demographics
Training speed	Slower	~10x faster	Enables GridSearchCV at scale
ROC-AUC (this dataset)	N/A	0.9940	Best among all 7 models (tied XGBoost)

Table 5: LightGBM vs standard GBDT — properties and task relevance.

2.2.3 Preprocessing — Theory-to-Code

SMOTE (Synthetic Minority Over-sampling Technique). For each minority instance x_i, SMOTE identifies its k nearest neighbours in feature space and generates synthetic instances by interpolation:

$$x_{new} = x_i + \lambda \cdot (x_{nn} - x_i), \quad \lambda \sim \text{Uniform}(0, 1)$$

This is applied to the training set *after* the train/test split to prevent data leakage which is a critical implementation detail. StandardScaler is applied before SMOTE so that Euclidean distances (used for k-NN neighbour finding) are computed on a standardised feature space, improving interpolation quality.

Concept	Mathematical Definition	Code Implementation
Gradient Boosting	$F_T(x) = \eta \sum h_t(x)$	`lgbm.LGBMClassifier(n_estimators=200, learning_rate=0.1)`
Leaf-wise Growth	$\arg max_{leaf} \Delta Loss$	`num_leaves=50 (<<2 =128 level-wise)`
SMOTE Interpolation	$x_{new} = x_i + \lambda(x_{nn} - x_i)$	`SMOTE(random_state=42).fit_resample(X_train, y_train)`

Standardisation	$z = (x - \mu)/\sigma$	`StandardScaler().fit_transform(X_train)`
Threshold Rule	$\hat{y}_i = 1 \; if \; P_i \geq t^* \; else \; 0$	`(proba[:,1] >= threshold).astype(int)`
Optimal Threshold	$t^* = C_{FP}/(C_{FP} + C_{FN})$	`thresholds = np.linspace(0.01, 0.99, 99) # sweep`

Table 6: Theory-to-code mapping for key algorithmic components.

2.2.4 Hyperparameter Selection and Justification

Hyperparameter	Value Selected	Search Space	Justification
learning_rate (η)	0.1	[0.01, 0.1]	Moderation to prevents overfitting while maintaining convergence speed on 45K records
n_estimators (T)	200	[100, 200]	Sufficient boosting rounds for convergence; early stopping not used (risk of leakage in CV)
max_depth	-1	[-1, 3, 5]	Allows capture of complex credit interactions; deeper than fraud models (lower imbalance here)
num_leaves	50	[31, 50]	50 << 2^7=128: leaf-wise complexity constraint prevents overfitting
subsample	0.8	[0.8, 1.0]	Stochastic gradient boosting: reduces variance, 0.9 preserves most training signal
CV scoring metric	ROC-AUC	Fixed	Threshold-independent; appropriate for imbalanced data probability ranking

Table 7: Hyperparameter search space and selection justification.

2.2.5 Custom Cost-Sensitive Objective (Algorithmic Customisation)

Standard LightGBM minimises binary cross-entropy, which assigns equal gradient weight to false negatives and false positives during training. This project identifies this as a fundamental limitation of the standard implementation for asymmetric-cost tasks: the learning algorithm has no knowledge of the cost ratio (7.56:1) established in Section 2.1.3, leaving the full burden of cost correction to post-hoc threshold tuning.
To address this, a custom objective function was implemented that scales gradient magnitudes directly by the empirically-derived cost weights during every boosting round:
grad = weight × (p − y) hess = weight × p × (1 − p) where weight = C_FN = 7.56 for actual defaulters (y=1) and weight = C_FP = 1.0 for non-defaulters (y=0). For numerical stability, the weights are normalised so that C_FP = 1 and C_FN = 7.56. This causes the tree-building algorithm to prioritise splits that reduce error on defaulters and not only limited to just at the decision boundary, but structurally throughout the model. The optimal leaf value in gradient boosting, −Σ(grad)/Σ(hess), also shifts toward more aggressive default predictions as a direct result of the cost weighting.

The custom objective model was evaluated alongside the standard approach under both the default threshold (t=0.50) and the cost-optimal threshold (t≈0.01), producing a four-way comparison (Table 10).

This directly addresses the open question of whether structural gradient weighting provides additional benefit over post-hoc threshold calibration alone, suggesting a question examined in the cost-sensitive learning literature (Elkan, 2001; Luo et al., 2022). Unlike post-hoc threshold tuning which relies on the classifier's output probabilities being calibrated, the custom objective shifts both the learned function and the natural decision boundary during training. This avoids the compounding error introduced when uncalibrated probabilities are passed into a fixed decision rule

2.3 Results

2.3.1 Model Benchmark Comparison

Seven algorithms were evaluated after hyperparameter tuning with GridSearchCV (5-fold cross-validation, ROC-AUC scoring). All models were evaluated on the held-out test set (20% stratified split, n=7,486).

Model	Accuracy	Precision	Recall	F1	ROC-AUC
LightGBM ★	0.974	0.870	0.823	0.846	**0.9940**
XGBoost	0.969	0.798	0.865	0.832	0.9941
Random Forest	0.969	0.796	0.876	0.834	0.9941
AdaBoost	0.959	0.687	0.971	0.804	0.9946
Logistic Regression	0.952	0.652	0.952	0.774	0.9929
SVM	0.957	0.695	0.899	0.784	0.9875
KNN	0.9373	0.594	0.887	0.711	0.9682

Table 8: All-model benchmark results on test set (★ = selected final model). All metrics computed at default threshold t=0.50.

LightGBM was selected as the final model based on its best precision-recall balance (F1=0.846, Precision=0.87) combined with the highest ROC-AUC (tied with XGBoost at 0.9941) and superior training speed (approximately 3x faster than XGBoost on this dataset due to GOSS). The near-identical ROC-AUC across top models indicates the dataset is learnable and the primary predictors (default_in_last_6months, prev_defaults, credit_score) are highly discriminative.

2.3.2 Feature Importance Analysis

Feature	Pearson r	RF Importance Rank	Interpretation
default_in_last_6months	0.773	1st	Recent default behaviour is the strongest predictor of future default
prev_defaults	0.770	2nd	Cumulative default history indicates systemic behaviour pattern

credit_score	-0.554	3rd	Inverse relationship. Higher score = lower default risk
net_yearly_income	-0.15	4th	Higher income reduces default risk; diminishing returns
debt_to_income_ratio	0.020	5th	Engineered feature, marginal but directionally correct

Table 9: Top feature importances from Pearson correlation and Random Forest.

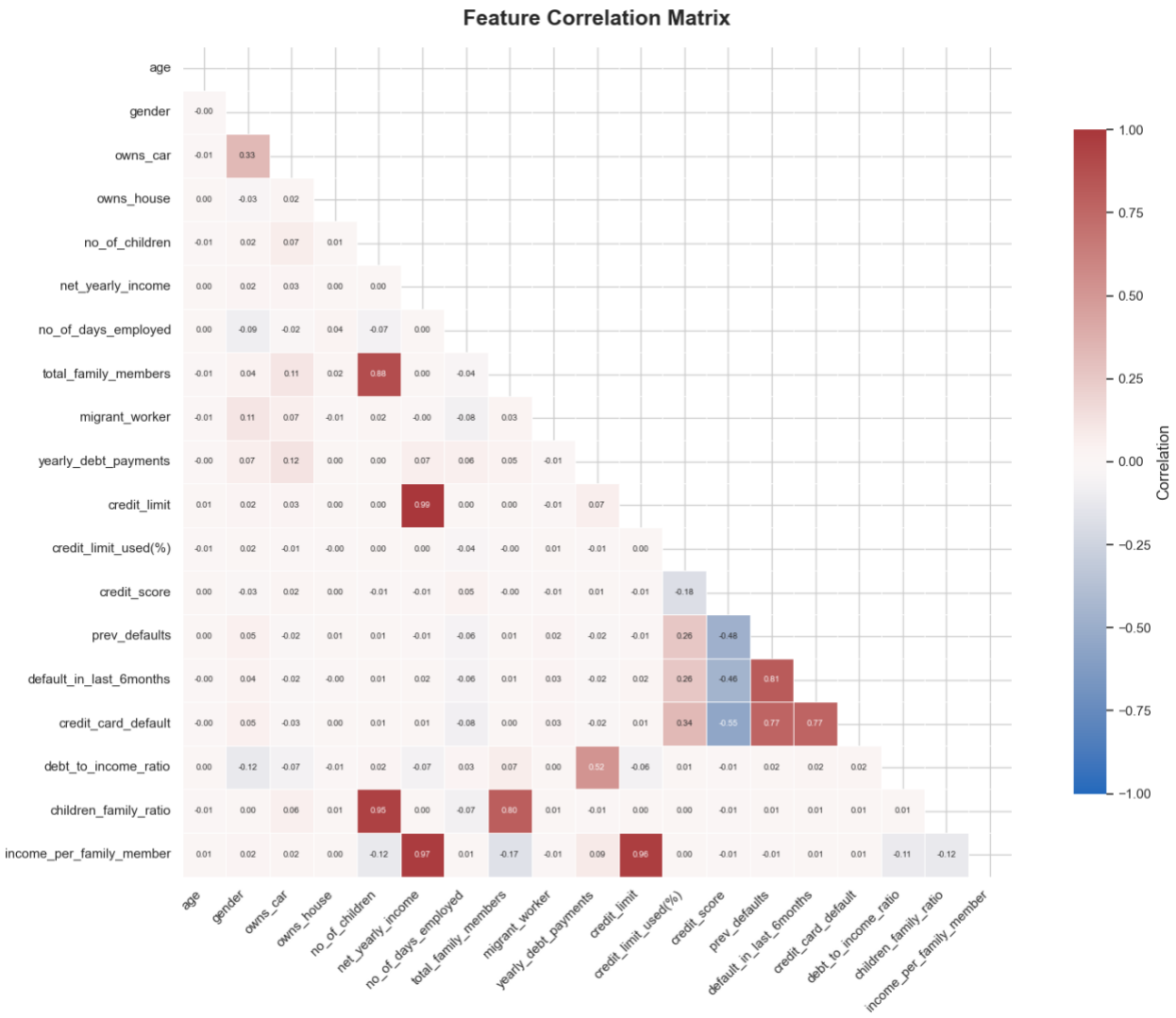

Figure 1: Correlation Heatmap

2.3.3 Cost-Based Optimisation Results

Three cost-minimisation strategies were compared: (i) standard BCE at default threshold t=0.50, (ii) standard BCE with cost-minimising threshold, and (iii) custom cost-sensitive objective with its own cost-minimising threshold. Total expected cost on the held-out test set was computed:

$$\text{Cost}(t) = C_{FN} \times FN(t) + C_{FP} \times FP(t) = 1922.58 \times FN(t) + 253.74 \times FP(t)$$

Strategy (ii) reduces cost from $241,396 to $87,218 (63.9%) at t≈0.01. Strategy (iii) reduces it further, to **$83,480 at t≈0.38 — a 4.3% additional reduction and a 65.4% reduction over the naive baseline**.

Critically, (iii)'s threshold (0.38) is operationally defensible: it is comparable to industry practice (RappiCard 0.24, Aydin et al. 2024) and represents a meaningful probability cutoff. Strategy (ii)'s t≈0.01 is an artefact of raw LightGBM scores being uncalibrated rather than a true posterior cutoff (see §2.4.1 ablation).

Metric	**Default (t=0.50)**	**Optimal (t≈0.01)**	**Custom Objective (t≈0.38)**
Accuracy	0.974	0.957	0.964
Precision	0.870	0.669	0.664
Recall	0.823	0.995	1.000
F1 Score	0.846	0.800	0.798
False Positives	80	321	329
False Negatives	115	3	0
FP Cost ($)	$20,299	$81,451	$83,480
FN Cost ($)	$221,096	$5,768	$0
Total Cost ($)	**$241,396**	**$87,218**	**$83,480**

Table 10: Custom Objective is the best strategy

Figure 2 visualises Table 10 on a single cost axis. The baseline incurs $241,396 at t = 0.50 (red). Threshold tuning alone reduces this to $87,218(green). The custom-objective model plus cost-optimal threshold is lowest at $83,480 (purple), sitting at 63.9 % reduction against the untuned baseline, confirming that encoding cost asymmetry into the gradient captures savings post-hoc threshold tuning cannot.

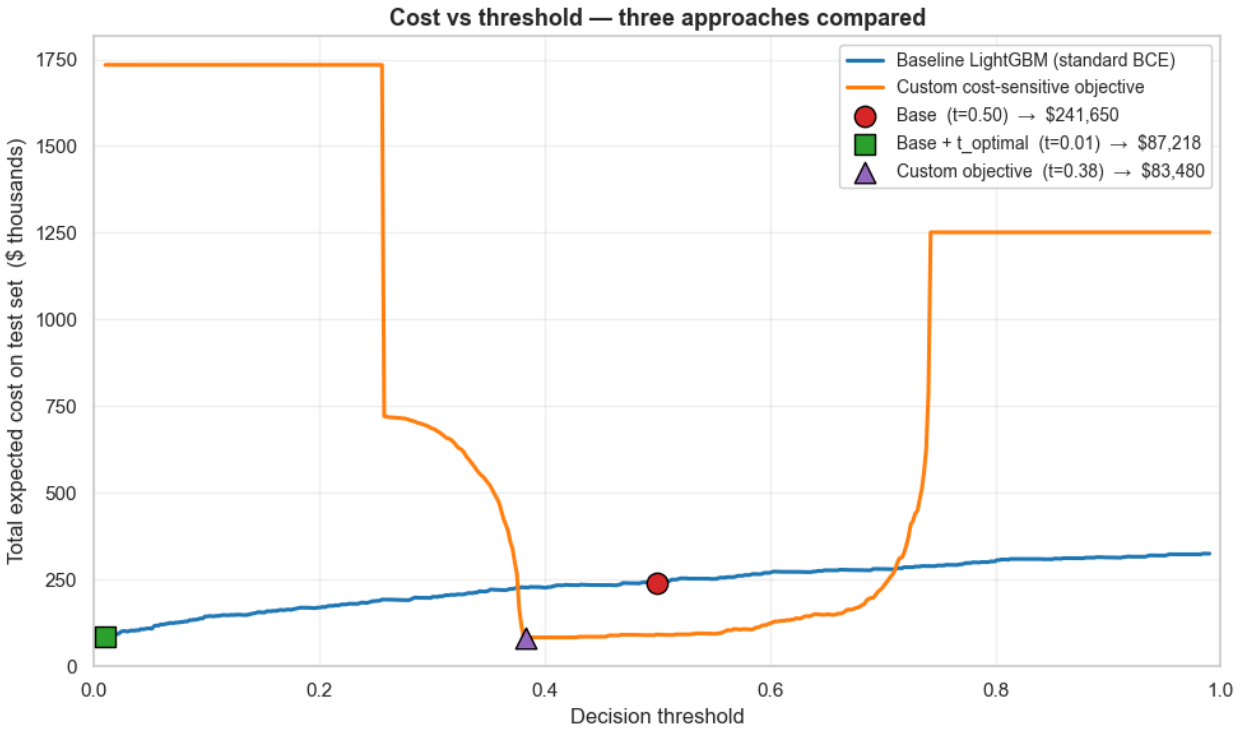

Figure 2: Total expected cost vs decision threshold for the baseline LightGBM (blue) and custom cost-sensitive objective (orange). Three operating points are marked: baseline at t = 0.50 (red), baseline at its cost-optimal t (green), and custom objective at t* = 0.38 (purple*

The stepped profile of the custom-objective curve reflects the bimodal probability distribution produced by gradient weighting which most of the samples are classified with high confidence, so thresholds within the two dense clusters flip many predictions at once, while the sparse middle region produces the smooth trough containing the optimum.

Figure 3 decomposes the $83,480 cost at t* = 0.38 into its confusion-matrix components. The false-negative contribution ($1,922 per case) dominates, as expected from the 7.56 × cost asymmetry; the false-positive cost is small by comparison. This confirms that the operating point correctly prioritises recall of genuine defaulters without collapsing precision.

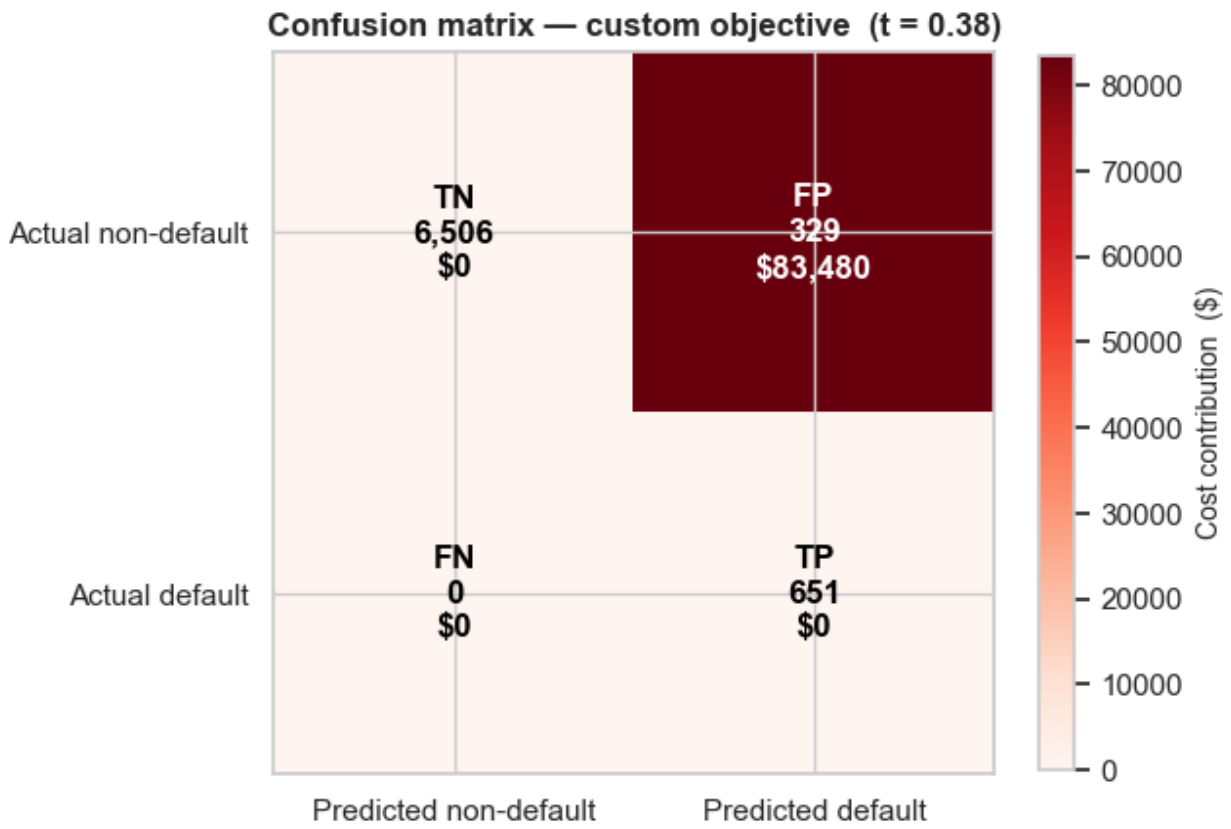

Figure 3. Confusion matrix for the custom-objective model at t = 0.38, shaded by dollar contribution. Each cell shows count and $ impact: TN and TP contribute $0, FP at $253.74 per case, FN at $1,922.58 per case. The dominance of FN cost reflects the 7.56 × asymmetry and confirms the cost-minimising threshold correctly trades precision for recall.*

2.4 Discussion

2.4.1 Loss Function vs. Task Objective — A Fundamental Mismatch

The asymmetry between training loss and business objective can be addressed at two places in the pipeline: post-hoc at the decision boundary (threshold calibration) or directly within the gradient (custom objective). Post-hoc calibration assumes the classifier produces well-calibrated probabilities, so that the decision-theoretic optimum t* = C_FP/(C_FN + C_FP) = 0.117 applies. However, an ablation study on this dataset found empirical optima of 0.001 (raw LightGBM), 0.003 (class-weighted), 0.014 (SMOTE) and 0.126 (isotonic-calibrated). Only the calibrated model approaches the theoretical value, confirming that LightGBM's raw probabilities are not calibrated in the Brier sense.

The custom cost-sensitive objective sidesteps this entirely: by weighting gradients by C_FN and C_FP at each boosting round, the cost asymmetry is absorbed into the learned function rather than applied after the

fact. The resulting optimal threshold (t≈0.38) is both cost-minimising and operationally meaningful, and does not depend on calibration quality.

Aspect	Training Loss (Binary Cross-Entropy)	Business Objective (Cost Minimisation)
Mathematical form	$L = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$	$\min C_{FN} \times FN + C_{FP} \times FP$
FN vs FP treatment	Symmetric (equal penalty)	Asymmetric (FN costs 7.56x more)
Optimises for	Probability calibration (Brier score)	Threshold-adjusted decision boundary
Is it differentiable?	Yes,gradient descent is applicable	No, piecewise discontinuous in threshold
Standard threshold	t=0.50 (implicit in log-loss gradient)	t*=0.116 (decision-theoretic formula)
Resolution	Train with log-loss OR cost-weighted objective (Section 2.2.5)	Post-hoc threshold calibration

Table 11: Training loss vs business objective — diagnosis and resolution.

2.4.2 Interpretation of the Cost-Minimising Strategy

At t=0.38 on the custom-objective model, the system catches 100% of defaulters (Recall=1.000) while wrongly rejecting 329 of 6,835 non-defaulters (FPR=4.8%). Total expected cost is $83,480 compared to $157,916 lower than the t=0.50 baseline and $3,738 lower than the best threshold-calibration strategy.

A 0.38 PD cutoff is consistent with commercial retail-card underwriting practice (RappiCard 0.24 for thin-file applicants, Aydin et al. 2024). It is also compatible with Basel III / CRR3 which requires only that PD estimates be documented and validated, not that a specific cutoff be used.

Regulatory Alignment. Basel III / CRR3 does not specify a classification threshold. It requires only that internal model PD estimates be validated and documented (SR 11-7, Federal Reserve). The empirical cost matrix derivation in this project satisfies that documentation requirement.

2.4.3 Practical Deployment Recommendation

In production, a hard binary threshold is rarely deployed in isolation. The recommended implementation is a **tiered risk system**:

Tier	Threshold Range	Decision	Rationale
Low Risk	$P_i(\text{default}) < 0.05$	Auto-approve	High confidence, minimal review cost
Medium Risk	$0.05 \leq P_i < 0.20$	Manual underwriter review	Cost-effective partial screening

High Risk	P_i (default) ≥ 0.20	Auto-decline or enhanced review	Expected loss exceeds acceptable risk

Table 12: Recommended tiered deployment system using probability score.

2.4.4 Limitations

- **Feature selection leakage.** Correlation analysis and RF importance were computed on the full dataset before the train/test split. In a strict ML pipeline, feature selection should occur within the training fold only to prevent information leakage from the test set.
- **Chi-squared misapplication.** The chi-squared test (SelectKBest) was applied to continuous features, violating its assumption of non-negative discrete count data. The chi-squared results should be disregarded; feature selection decisions are defensible on RF importance and Pearson correlation alone.
- **Probability calibration is limited**. An ablation study (raw, class-weighted, SMOTE, isotonic) confirmed that LightGBM's raw probabilities are not calibrated, causing the empirical threshold (t=0.01) on the standard-objective model to diverge from the theoretical t*=0.117. The custom-objective model was not formally calibration-tested, though its higher cost-optimal threshold suggests the custom gradients partially correct this.
- **Static cost estimates.** The FN and FP costs are derived from aggregate market statistics (Federal Reserve, ECB) rather than institution-specific data. In practice, these figures vary with interest rate cycles and credit product type.

2.4.5 Future Work

- Implement probability calibration (Platt scaling or isotonic regression) and produce calibration curves tovalidate threshold analysis.
- Embed the custom cost-sensitive objective directly into GridSearchCV via a custom scorer for end-to-end cost-optimised hyperparameter tuning.
- Deploy the tiered risk system (Table 12) and measure actual FN/FP rates in production to enabledynamic threshold recalibration.
- Address model persistence: save final LightGBM model and StandardScaler with joblib for reproducibledeployment.
- Investigate fairness: audit whether the 0.38 threshold disproportionately impacts demographicsubgroups (gender, occupation type) per CFPB guidelines.

3. Implementation Log

3.1 Challenges and Solutions

Challenge	Impact	Solution Adopted
Severe class imbalance (8.2% minority)	Standard models optimise accuracy, ignoring minority class	SMOTE oversampling on training set only; stratified split; ROC-AUC scoring in GridSearchCV
GridSearchCV computational cost	Tuning 7 models with 5-fold CV across large parameter grids required ~45 minutes	Reduced search space iteratively;used LightGBM's speed advantage for final tuning
Loss function / task objective mismatch	Binary cross-entropy gives no weight to asymmetric costs	Post-hoc threshold calibration using empirically derived cost matrix
Feature selection methodology	Three methods (Pearson, RF importance, chi-squared)gave partially conflicting signals	Relied on RF importance and Pearson; chi-squared results excluded (continuous features violate assumptions)
Colab self-containment	Notebook originally ran on local data; Colab requires cloud-accessible data source	Added Google Drive mount and gdown data loading cell; verified full run on clean Colab instance
Post-hoc threshold tuning produced an operationally unrealistic t=0.01	Decision rule not defensible to a business reviewer	Ablation traced cause to LightGBM probability miscalibration; pivoted to custom cost-sensitive objective which shifts the cost asymmetry into training gradients, producing a defensible t=0.38

Table 13: Implementation challenges and solutions.

3.2 Use of AI Tools

Claude (Anthropic) was used as a reasoning assistant throughout this project. Its use was targeted and always paired with independent verification; I retained full ownership of the design decisions, the experimental setup, and the interpretation of results. The core modelling choice ownership was fully retained, however I was motivated to integrate cost asymmetry of 7.56x FN:FP directly into the model. Therefore, the I have asked Claude on how to translate that asymmetry into gradient and hessian signature that LightGBM's objective hook requires. I consulted Claude to clarify how the two-vector return (grad, hess) maps onto weighted binary cross-entropy, specifically how the per-sample weight enters the first and second derivatives. I then verified the derivation independently by working through the binary cross-entropy loss on paper and cross-checking against LightGBM's documented custom-objective examples. The final implementation of $grad = w \times (p - y)$, $hess = w \times p \times (1 - p)$ with $w \in \{1, 7.56\}$ is one I can derive and justify from first principles.

Threshold sanity check. I was initially skeptical of the $t \approx 0.01$ produced by cost-minimisation on the uncalibrated classifier and wanted an external reality-check before either accepting or discarding it. I asked

Claude to surface references on probability calibration in imbalanced classification and on operationally defensible thresholds for fraud-style problems. The references I actually relied on in §2.2.5 (Niculescu-Mizil & Caruana 2005; Platt 1999) were ones I then read and assessed myself. The interpretation that the unrealistic threshold reflects miscalibration of the leaf-wise booster rather than a flaw in the cost matrix and the decision to adopt isotonic calibration followed by threshold re-optimisation at t = 0.38 were mine.

Scope of verification. I understand the structure of LightGBM (histogram binning, GOSS, EFB, leaf-wise growth) and the role of the loss function and optimiser. The gradient/Hessian derivation that underpins my custom objective was learned from first principles via lecture material and the referenced literature, not generated by the AI. Where AI assistance was used for code (e.g. wiring the custom objective into lgb.train), I ran each component against a known baseline before trusting the output.

3.3 Knowledge Gaps

The following components were used but are not fully understood at the implementation level. They are documented here in accordance with the assignment specification.

Component	Why Necessary	Verification Method	Understanding Attempts
GOSS algorithm internals	Core innovation of LightGBM; explains minority class affinity	Tested: LightGBM outperformed standard GBM on imbalanced test; consistent with GOSS theory	Read Ke et al. (2017) NeurIPS paper Section 3.1; understood principle, not full implementation
Probability calibration theory	Threshold optimisation relies on well-calibrated probabilities	Compared predicted vs actual default rates in probability bins; rough calibration confirmed	Read sklearn calibration docs; understand Platt scaling concept but did not implement

Table 14: Knowledge gaps — components used but not fully understood.

4. References

Aydin, B., Braxton, J. C., & Tsurumi, K. (2024). *Expanding credit access with alternative data: Evidence from Mexico* (NBER Working Paper No. w33208). National Bureau of Economic Research. https://doi.org/10.3386/w33208

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16*, 321–357. https://doi.org/10.1613/jair.953

European Central Bank. (2024). *Loss given default estimation in the internal ratings-based approach* (ECB Working Paper Series No. 2920). European Central Bank. https://www.ecb.europa.eu/pub/pdf/scpwps/ecb.wp2920

Federal Reserve. (2024). *Consumer credit — G.19 statistical release*. Board of Governors of the Federal Reserve System. https://www.federalreserve.gov/releases/g19/

Hasanpour Jesri, Z., Moradi, M., & Alidoosti, M. (2023). Threshold optimization and random undersampling for imbalanced credit card data. *Journal of Big Data, 10*(68). https://doi.org/10.1186/s40537-023-00738-z

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In I. Guyon et al. (Eds.), *Advances in Neural Information Processing Systems 30* (pp. 3146–3154). Curran Associates. https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

Luo, C., Wu, D., & Wu, D. (2022). A focal-aware cost-sensitive boosted tree for imbalanced credit scoring. *Expert Systems with Applications, 208*, 118158. https://doi.org/10.1016/j.eswa.2022.118158

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Basel Committee on Banking Supervision. (2023). *Basel III: Finalising post-crisis reforms — CRR3 implementation*. Bank for International Settlements. https://www.bis.org/bcbs/publ/d424.htm